

Memory Link PWA

가족의 추억을 할아버지께 전하는 슬라이드쇼 앱

작성일

2026년 4월 26일

배포 URL

ohseonghwan538.github.io/memory-link

GITHUB

Ohseonghwan538 / memory-link

FIREBASE 프로젝트

memory-link-5a665 (서울)

목차

1. 프로젝트 개요
2. 파일 구조
3. 기술 스택 및 아키텍처
4. 화면 구조 (View 시스템)
5. 데이터 저장 구조 (3중 레이어)
6. 사진 처리 파이프라인
7. Firebase 실시간 동기화
8. PWA 기술 상세
9. 인프라 구성 및 배포 흐름
10. 보안 현황 및 한계
11. 무료 한도 모니터링
12. 유지보수 가이드

프로젝트 개요

가족 구성원이 사진과 글을 올리면 멀리 계신 할아버지가 슬라이드쇼 형태로 감상할 수 있는 PWA(Progressive Web App)입니다. 별도 앱 설치 없이 링크 하나로 모든 기기에서 동작하며, 홈 화면에 추가하면 일반 앱처럼 실행됩니다.

항목	내용
앱 형태	PWA (Progressive Web App)
서버 코드	없음 — 100% 클라이언트 사이드
호스팅	GitHub Pages (무료 정적 CDN)
데이터베이스	Firebase Firestore (무료 Spark 플랜)
빌드 도구	없음 — 순수 HTML/CSS/JS
월 운영비용	0원 (무료 한도 내)

파일 구조

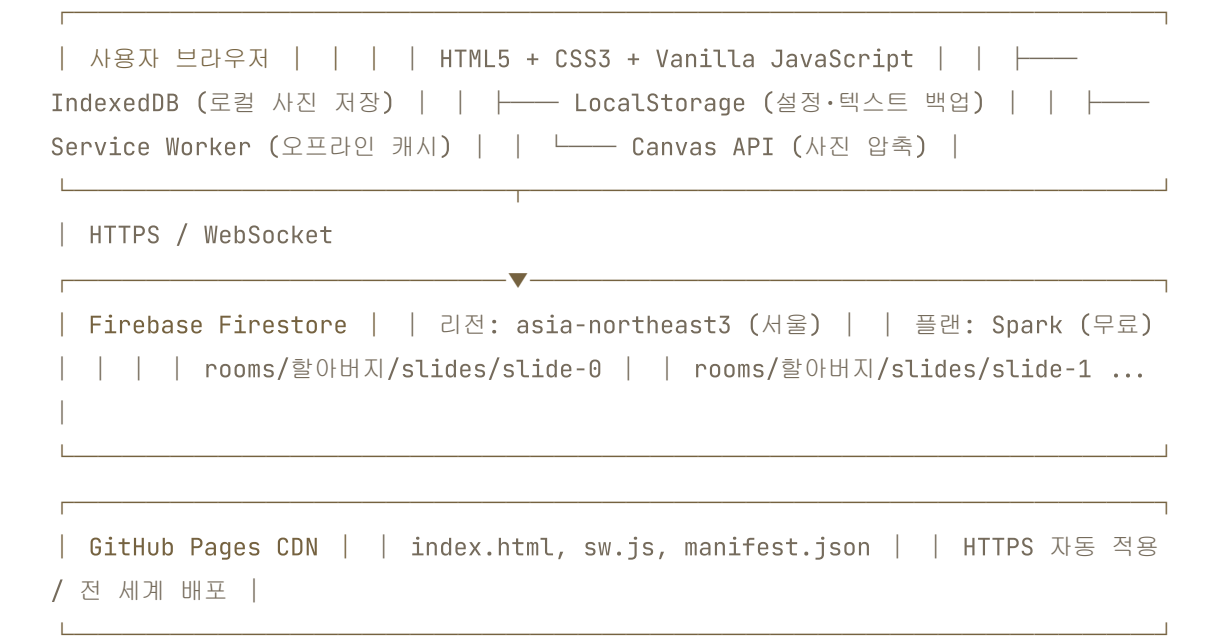
전체 앱이 4개의 파일로 구성됩니다. React, Vue 등의 프레임워크를 사용하지 않아 빌드 과정 없이 브라우저가 직접 실행합니다.

memory-link-app/

- └── index.html (40KB) 앱 전체 - HTML + CSS + JS 통합 단일 파일
- └── manifest.json (569B) PWA 설치 정보 (앱 이름, 아이콘, 시작 URL)
- └── sw.js (1.4KB) Service Worker - 오프라인 캐시 담당
 - └── icons/
 - └── icon-192.png Android 홈화면 아이콘
 - └── icon-512.png 스플래시 스크린용 아이콘
 - └── icon.svg 원본 벡터 아이콘

파일	역할	크기
index.html	앱의 모든 것 (UI 구조, 스타일, 로직)	40KB
manifest.json	PWA 메타데이터 - 브라우저가 읽어 설치 기능 제공	569B
sw.js	Service Worker - 별도 스레드에서 캐시 관리	1.4KB

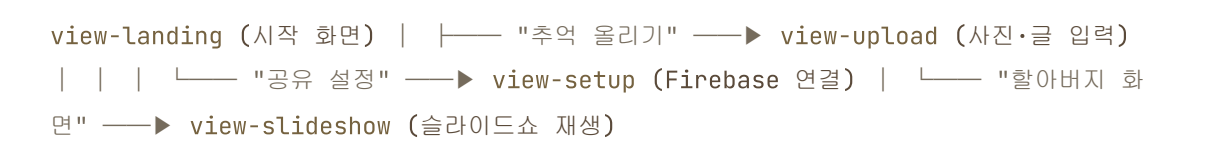
기술 스택 및 아키텍처



기술	용도	비고
Vanilla JS (ES5)	앱 전체 로직	구형 기기 호환성을 위해 ES5 문법 사용
IndexedDB	로컬 사진 저장	용량 제한 없음 (기기 여유공간 기준)
LocalStorage	텍스트 백업	IndexedDB 실패 시 fallback, 5MB 한도
Canvas API	사진 압축/리사이즈	최대 800px, JPEG 70% 품질
Service Worker	오프라인 캐시	Cache First 전략
Web App Manifest	PWA 설치 정보	홈 화면 추가 기능 제공
Firebase Firestore	실시간 데이터 동기화	onSnapshot으로 WebSocket 기반 수신
Firebase SDK v10.12	Firestore 클라이언트	CDN에서 compat 버전 로드
GitHub Pages	정적 파일 호스팅	git push 시 자동 배포

화면 구조 (View 시스템)

별도 라우터 없이 CSS `display` 속성 토글로 화면을 전환합니다. 4개의 View가 항상 DOM에 존재하며, `active` 클래스가 있는 View만 표시됩니다.



View ID	역할	주요 기능
view-landing	시작 화면	가족용 / 할아버지용 분기
view-upload	업로드 화면	사진 선택, 텍스트 입력, 클라우드 공유
view-slideshow	슬라이드쇼	자동 재생 (10초), 수동 이동, 진행 바
view-setup	공유 설정	Firebase 원탭 연결

데이터 저장 구조 (3중 레이어)

데이터는 로컬 2단계 + 클라우드 1단계의 총 3중 구조로 관리됩니다.

사진 업로드 시 ↓ [1단계] IndexedDB (기기 로컬) - 사진(base64) + 텍스트 전체 저장 -
앱 재시작 시 자동 복원 ↓ 실패 시 (구형 브라우저) [2단계] LocalStorage (기기 로컬 백업)
- 텍스트만 저장 (사진 제외, 5MB 한도) ↓ "슬라이드쇼 시작" 클릭 시 [3단계] Firebase
Firestore (클라우드) - 사진 압축 후 base64로 저장 - 모든 가족 기기에 실시간 동기화

Firestore 데이터 구조

```
rooms (컬렉션)
├─ 할아버지 (문서 - 룸 코드)
│   ├─ lastActive: 1714123456789
│   ├─ agent: "Mozilla/5.0 ..."
│   └─ slides (서브컬렉션)
│       ├─ slide-0
│       │   ├─ order: 0
│       │   ├─ photo: "data:image/jpeg;base64,/9j/4AAQ..."
│       │   ├─ when: "2024년 봄, 경주 여행"
│       │   ├─ who: "할머니, 아빠(7살)"
│       │   └─ story: "그날 할아버지께서..."
│       ├─ slide-1 { ... }
│       └─ slide-2 { ... }
```

사진 처리 파이프라인

Firebase Storage가 유료 플랜(Blaze)만 지원하도록 정책이 변경되어, Canvas API로 사진을 압축한 뒤 Firestore에 직접 저장하는 방식을 사용합니다.

```
[원본 사진 선택] 사용자가 파일 탐색기에서 선택 ↓ FileReader.readAsDataURL() [base64 문자열로 변환] 기기 메모리(IndexedDB)에 보관 ↓ "슬라이드쇼 시작" 클릭 → compressPhoto() 호출 [Canvas 리사이즈] - 긴 변 기준 최대 800px 축소 - canvas.toDataURL('image/jpeg', 0.70) ↓ [압축된 base64 → Firestore 저장]
```

원본	압축 후	압축률
iPhone 사진 (4MB)	약 150~300KB	약 93% 감소
일반 카메라 (8MB)	약 200~400KB	약 95% 감소
스크린샷 (500KB)	약 50~100KB	약 80% 감소



Firestore 문서 1개의 크기 한도는 **1MB**입니다. 압축 후 사진이 약 400KB 이하여야 안정적으로 저장됩니다. 매우 고해상도 사진(12MP 이상)은 압축률을 더 낮추거나 해상도 상한을 낮추는 조정이 필요할 수 있습니다.

Firestore 실시간 동기화

업로드 흐름 (가족 → 클라우드)

슬라이드쇼 시작 버튼 클릭 → 기존 slides 서브컬렉션 전체 삭제 (batch.delete) → 사진 압축 (Promise.all) → 새 슬라이드 전체 재업로드 (col.doc('slide-N').set(...)) → Firestore가 변경 감지 → onSnapshot 콜백 실행 → 할아버지 화면 자동 갱신

수신 흐름 (클라우드 → 할아버지)

```
// Firebase 연결 시 1회 등록 - 이후 자동 실행
fbDb.collection('rooms').doc('할아버지')
  .collection('slides').orderBy('order')
  .onSnapshot(function(snapshot) {
    // 변경 발생 시 Firestore가 자동으로 이 콜백 실행
    // (폴링 없음 - WebSocket 기반 서버 push)
    slides = snapshot.docs.map(doc => doc.data());
    saveToStorage(); // 로컬에도 저장
    showToast('📺 가족이 추억을 업데이트했어요');
  });
```



WebSocket 기반으로 동작하므로 주기적 새로고침 없이 실시간 수신됩니다. 가족이 사진을 올리면 할아버지 화면에 수 초 내에 반영됩니다.

PWA 기술 상세

Service Worker (sw.js) — Cache First 전략

요청 발생 ↓ 캐시에 있음? → 캐시에서 즉시 반환 (오프라인도 가능) 캐시에 없음? → 네트워크
요청 → 성공 시 캐시에 저장 네트워크도 실패? → index.html 반환 (앱 껌데기라도 표시)

캐시 키는 'memory-link-v2' 입니다. 앱 코드를 업데이트할 때 이 버전 번호를 올리면 구버전 캐시가 자동 삭제됩니다.

iOS vs Android 설치 차이

	iOS (Safari)	Android (Chrome)
설치 방법	공유 버튼 → 홈 화면에 추가	하단 배너 또는 메뉴 → 앱 설치
브라우저 제약	Safari만 가능	모든 Chromium 브라우저
감지 방법	navigator.standalone	beforeinstallprompt 이벤트
실행 결과	주소창 없이 앱처럼 실행	주소창 없이 앱처럼 실행

인프라 구성 및 배포 흐름

항목	서비스	세부 정보
코드 저장소	GitHub	github.com/Ohseonghwan538/memory-link
호스팅	GitHub Pages	master 브랜치 / root 경로 자동 배포
도메인	GitHub 기본 도메인	ohseonghwan538.github.io/memory-link/
HTTPS	GitHub 자동 적용	별도 설정 불필요
데이터베이스	Firebase Firestore	memory-link-5a665 / 서울 리전
Firebase 플랜	Spark (무료)	Storage 미사용 — Firestore만 활용

배포 흐름

index.html 수정 (로컬) → `git add index.html` → `git commit -m "변경 내용"` → `git push` → GitHub Actions 자동 실행 (약 1~2분) → ohseonghwan538.github.io/memory-link/ 에 반영

GitHub 커밋 이력

커밋	내용
0f8e490	Initial commit: Memory Link PWA
982d9c5	Firebase Storage → Firestore 직접 저장, 사진 압축 추가
8b884e0	Firebase 설정 내장, 원탭 연결로 UX 단순화, 룸코드: 할아버지

보안 현황 및 한계

항목	현재 상태	위험도
Firebase API Key 노출	코드에 포함 (공개 저장소)	낮음 클라이언트 앱의 표준 방식
Firestore 보안 규칙	테스트 모드 (30일 한시)	주의 필요 만료 후 전체 차단
사용자 인증	없음 (룸 코드만)	낮음 가족 내부 앱
HTTPS	적용됨	안전



Firestore 테스트 모드 만료일: 2026년 5월 25일
이 날짜 이후에는 데이터 읽기·쓰기가 전면 차단됩니다. 아래 보안 규칙으로 업데이트해야 합니다.

만료 전 적용할 Firestore 보안 규칙

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /rooms/{roomId}/{document=**} {
      allow read, write: if true;
    }
  }
}
```

적용 위치: Firebase Console → memory-link 프로젝트 → Firestore Database → 규칙 탭 → 게시

SECTION 11

무료 한도 모니터링

서비스	무료 한도	가족 예상 사용량	여유
GitHub Pages 트래픽	100GB / 월	수십 MB / 월	여유 충분
Firestore 읽기	50,000건 / 일	수십 건 / 일	여유 충분
Firestore 쓰기	20,000건 / 일	수 건 / 일	여유 충분
Firestore 저장 용량	1GB	압축 사진 수백 장 = 수십~수백 MB	여유 충분



가족 4~5명 기준 일 사용량은 무료 한도의 0.1% 수준입니다. 사실상 평생 무료로 운영 가능합니다.

유지보수 가이드

앱 비활성화 / 재활성화

방법	절차	효과
GitHub Pages 끄기	저장소 Settings → Pages → Unpublish site	URL 접근 불가 (404)
GitHub Pages 켜기	같은 페이지에서 Save 클릭	1~2분 후 복구
Firestore만 차단	보안 규칙을 allow: if false 로 변경	UI는 보이되 공유 불가

앱 업데이트 방법

```
# 1. index.html 수정 후
git add index.html
git commit -m "업데이트 내용"
git push
# → 1~2분 후 URL에 자동 반영
```

캐시 강제 갱신

앱 업데이트 후 사용자에게 새 버전이 로드되게 하려면 `sw.js` 의 캐시 버전 번호를 올려야 합니다.

```
// sw.js
const CACHE = 'memory-link-v3'; // v2 → v3 으로 변경
// 다음 접속 시 구버전 캐시 자동 삭제 후 새 버전 로드
```

문의 / 수정 요청

항목	정보
GitHub 저장소	github.com/Ohseonghwan538/memory-link
Firebase Console	console.firebase.google.com → memory-link-5a665
앱 URL	https://ohseonghwan538.github.io/memory-link/